

Laços de repetição e modularização

Programação Imperativa

Prof. Pedro Rocha Horchulhack

Contato: p.horchulhack@pucpr.br

Agenda

1. Estruturas de controle de fluxo
2. Laços de Repetição
3. Modularização

Agenda

1. Estruturas de controle de fluxo

2. Laços de Repetição

3. Modularização

Estruturas de controle de fluxo

Expressões lógicas

- Expressões cujos operadores são lógicos ou relacionais e cujos operandos são relações ou variáveis ou constantes do tipo lógico
- Operadores lógicos
 - E, OU e NÃO
- Como representamos em Rust?
 - E: `&&`
 - OU: `||`
 - NÃO: `!`

Estruturas de controle de fluxo

Expressões e operadores aritméticos

Operador	Símbolo	Símbolo de atribuição
Soma	+	+=
Subtração	-	-=
Multiplicação	*	*=
Divisão	/	/=
Resto (mod)	%	%=

Estruturas de controle de fluxo

Operadores *bitwise*

- Servem para realizarmos cálculos *bit a bit*;
- Operações **lógicas** *bit a bit*;
- Comumente utilizado para:
 - Otimização de performance;
 - Extração de *bits* específicos de um número;
 - Protocolos de rede.

Operador	Símbolo
E	&
OU	
OU exclusivo (XOR)	^
Negação	!

Estruturas de controle de fluxo

Operações *bitwise*

&

1	1	0	1	1	0	1	0
1	0	0	1	0	1	1	1
1	0	0	1	0	0	1	0

Estruturas de controle de fluxo

Operações *bitwise*

	1	1	0	1	1	0	1	0
	1	0	0	1	0	1	1	1
	1	1	0	1	1	1	1	1

Estruturas de controle de fluxo

Operações *bitwise*

^

1	1	0	1	1	0	1	0
1	0	0	1	0	1	1	1
0	1	0	0	1	1	0	1

Estruturas de controle de fluxo

Operadores lógicos

Operador	Símbolo
Igual a	==
Diferente de	!=
Maior que	>
Menor que	<
Maior ou igual a	>=
Menor ou igual a	<=

Estruturas de controle de fluxo

Controle de fluxo

```
fn main() {  
    let numero = 3;  
  
    if numero < 5 {  
        println!("condição verdadeira!");  
    } else {  
        println!("condição falsa...");  
    }  
}
```

Estruturas de controle de fluxo

Controle de fluxo

```
fn main() {  
    let numero = 3;  
  
    if numero < 5 {  
        println!("condição verdadeira!");  
    } else if numero == 3 {  
        println!("condição parcial...");  
    } else {  
        println!("Número qualquer!");  
    }  
}
```

Estruturas de controle de fluxo

1. Crie um programa em Rust que recebe como entrada a idade de um usuário e, se for maior de idade, mostre na tela que ele é maior de idade. Caso contrário, mostre que ele é menor de idade.
2. Crie um programa para autenticar um usuário. Solicite o nome de usuário e senha e, em seguida, verifique se o nome de usuário é "admin" e a senha é "@dm1n". Caso a senha ou o usuário não corresponderem, mostre na tela que o usuário ou a senha estão incorretos. Caso contrário, mostre que o usuário foi autenticado.
3. **[DESAFIO]** Crie um programa em Rust que faça o cálculo dos números pares da sequência de Fibonacci. Implemente o algoritmo de maneira **iterativa** (sem recursividade).

Agenda

1. Estruturas de controle de fluxo

2. Laços de Repetição

3. Modularização

Laços de Repetição

- É comum que queiramos repetir blocos de código;
 - Percorrer por uma sequência de dados;
 - Executar um programa até que uma condição seja atendida;
- Para isso utilizamos os laços de repetição;
- Na linguagem Rust existem três tipos:
 - `for`
 - `while`
 - `loop`

Laços de Repetição

for

- Percorre por elementos de uma sequência;
- Projetado, principalmente, para evitar *bugs* como:
 - Percorrer além do final do vetor;
 - Pular alguns elementos;
 - Desconsiderar itens faltantes;
- Não precisamos nos preocupar com a quantidade de elementos em uma sequência.

Laços de Repetição

Exemplo 1 – for

```
fn main() {  
    let a = [10, 20, 30, 40, 50];  
  
    for elemento in a {  
        println!("{}", elemento);  
    }  
}
```

Laços de Repetição

Exemplo 2 – for

```
fn main() {  
    let mensagem = "Programação";  
  
    for caracter in mensagem.chars() {  
        println!("{}", caracter);  
    }  
}
```

Importante lembrar

A função `.chars()` é específica para variáveis dos tipos `&str` ou `String`. O tipo retornado é uma estrutura conhecida como *iterador*.

Laços de Repetição

`while`

- São condicionais;
- Executam até com que uma condição seja satisfeita;
 - Implicitamente, chama o comando `break`;
 - Podemos explicitamente chamar o comando, caso queiramos parar a execução;

Laços de Repetição

Exemplo – while

```
fn main() {  
    let mut contador_regressivo = 5;  
  
    while contador_regressivo != 0 {  
        println!("Faltam {} segundos para decolar", contador_regressivo);  
        contador_regressivo = contador_regressivo - 1;  
    }  
    println!("Decolando!");  
}
```

Laços de Repetição

Exemplo – while

```
fn main() {  
    let mut contador_regressivo = 5;  
  
    while contador_regressivo != 0 {  
        if contador_regressivo == 1 {  
            println!("Falta {} segundo para decolar", contador_regressivo);  
        } else {  
            println!("Faltam {} segundos para decolar", contador_regressivo);  
        }  
        contador_regressivo = contador_regressivo - 1;  
    }  
    println!("Decolando!");  
}
```

Laços de Repetição

loop

- Executa o código **infinitamente**;
- A interrupção deve ser **explícita** (break);
- Ou é possível cancelar a execução pressionando CTRL+C.

```
fn main() {  
    loop {  
        println!("De novo!");  
    }  
}
```

Laços de Repetição

- Podemos **pular** iterações com o comando `continue`;
- Seu funcionamento é simples: `continue` para a próxima iteração.

```
fn main() {  
    let a = [10, 20, 30, 40, 50];  
  
    for elemento in a {  
        if elemento % 2 == 0 {  
            continue;  
        }  
        println!("{}", elemento);  
    }  
}
```

Laços de Repetição

- Podemos criar intervalos de números com objetos Range
- O exemplo abaixo cria a sequência de 0 até 9 ($n - 1$)

```
fn main() {  
    for i in 0..10 {  
        println!("{}", i);  
    }  
}
```

Laços de Repetição

Exemplo de iteração inversa

```
fn main() {  
    for i in (0..10).rev() {  
        println!("{}", i);  
    }  
}
```

Importante lembrar

A função `.rev()` é específica para tipos *iteradores*. Ela funciona como o próprio nome já diz: retorna a sequência inversa do *iterador*.

Laços de Repetição

Exercícios

1. Mostre os n elementos da sequência de Fibonacci. Solicite o valor de n para o usuário. Implemente o algoritmo utilizando o laço de repetição `while`.
2. Solicite uma `String` do usuário e mostre a quantidade de vogais e consoantes. Implemente o algoritmo usando o laço de repetição `for`.
3. Crie uma calculadora simples que permite operações entre dois números inteiros assinados de 32 bits solicitados do usuário. Mostre um menu ao usuário, solicitando a operação que deseja realizar. Implemente utilizando o laço de repetição `loop`. Considere as quatro operações básicas entre os dois números **e uma opção de sair do programa**:
 - Soma
 - Subtração
 - Divisão
 - Multiplicação
 - Sair
4. Calcule a soma dos números ímpares em um intervalo de 0 até 100. Implemente utilizando `while`.
5. Leia números inteiros do teclado indefinidamente. Quando digitado o valor 0, o programa deverá encerrar e mostrar a média dos números fornecidos de entrada.

Agenda

1. Estruturas de controle de fluxo

2. Laços de Repetição

3. Modularização

Modularização

- Até o momento só desenvolvemos nosso código no arquivo `main.rs`;
- Projetos crescem e demandam mais organização;
- Dividir o código em pequenos blocos, como funções, melhoram:
 - Legibilidade;
 - Manutenibilidade;
- Com muitas funções, em Rust organizamos elas em **módulos**

Modularização

- Um módulo consiste em um *namespace* que contém:
 - Definições de funções;
 - Definições de tipos;
- Podemos definir a visibilidade desses recursos como privado ou público;
 - Por padrão, **tudo** é considerado privado;
- Utilizamos a palavra chave `mod` para declaração de um módulo.

Modularização

Exemplo de módulo

- Definição de um módulo chamado `autenticacao`
- Três funções

```
mod autenticacao {  
    fn cadastrar() {}  
    fn autenticar() {}  
    fn redefinir_senha() {}  
}
```

Modularização

Exemplo de módulo – Acessando um componente do módulo

```
mod autenticacao {  
    fn cadastrar() {}  
    fn autenticar() {}  
    fn redefinir_senha() {}  
}  
  
fn main() {  
    autenticacao::cadastrar();  
}
```

Discuta com os colegas

Execute o código acima e discuta o resultado com os colegas. Por que deu erro? Como resolvemos?

Resposta

O erro acontece porque **todos** os membros são privados por padrão. Resolvemos colocando a palavra chave `pub` na frente da função.

Modularização

Exemplo de módulo – Acessando um componente do módulo

```
mod autenticacao {  
    pub fn cadastrar() {}  
    fn autenticar() {}  
    fn redefinir_senha() {}  
}  
  
fn main() {  
    autenticacao::cadastrar();  
}
```

Modularização

- É possível implementar vários módulos em um mesmo arquivo .rs

```
mod autenticacao {  
    pub fn cadastrar() {}  
    fn autenticar() {}  
    fn redefinir_senha() {}  
}
```

```
mod controle_de_acesso {  
    pub fn verificar_permissao() {}  
    fn conceder_permissao() {}  
}
```

```
fn main() {  
    autenticacao::cadastrar();  
    controle_de_acesso::verificar_permissao();  
}
```

Modularização

- Podemos definir módulos dentro de módulos

```
mod autenticacao {  
  pub fn cadastrar() {}  
  fn autenticar() {}  
  fn redefinir_senha() {}  
}
```

```
mod biometrica {  
  pub fn digital() {}  
  pub fn retina() {}  
}
```

```
mod senha {  
  pub fn otp() {}  
  pub fn senha_padrao() {}  
}
```

```
}
```

```
fn main() {  
  autenticacao::cadastrar();  
  autenticacao::biometrica::digital();  
  autenticacao::senha::otp();  
}
```

Modularização

Módulos dentro de módulos

main.rs

autenticacao

— cadastrar()

— autenticar()

— redefinir_senha()

— biometrica

— digital()

— retina()

— senha

— otp()

— senha_padrao()

Modularização

- Não é muito eficiente a declaração de módulos dentro de um mesmo arquivo;
- Podemos criar arquivos e diretórios para representar os módulos;
- Vamos implementar o exemplo abaixo.

```
autenticacao
```

```
|  cadastrar()  
|  autenticar()  
|  redefinir_senha()
```

Modularização

Prática 1

1. Abra o *terminal/cmd* em um diretório específico para essa prática;
2. Crie um novo projeto utilizando o cargo;
`cargo init pratica-modulos`
3. Abra o Visual Studio Code na pasta do projeto;
`code pratica-modulos`
4. No diretório `src` crie o arquivo `autenticacao.rs`;
5. No arquivo `autenticacao.rs` crie as três funções e torne-as públicas;
 - 5.1 `cadastrar()`
 - 5.2 `autenticar()`
 - 5.3 `redefinir_senha()`
6. Abra o arquivo `main.rs` e inclua o módulo `autenticacao`;
Adicione o trecho “`mod autenticacao;`” no início do código.
7. Chame as três funções implementadas.

Modularização

Prática 2

1. No mesmo projeto, crie o diretório calculadora;
2. No diretório calculadora, crie os seguintes arquivos:
 - operacoes.rs
 - menu.rs
 - mod.rs
3. Adeque o programa 3 criado no Slide 22 ([Clique aqui para visualizar o enunciado](#)). Organize-o nos módulos criados.